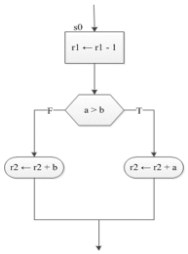


ТЕМА 4
МОДЕЛИРАНЕ НА ЦИФРОВИ СХЕМИ НА РЕГИСТРОВО НИВО

Цел:

Придобиване на практически опит за съставяне на VHDL модели на цифрови схеми на регистрово ниво.

I. Теоретична част

На регистрово ниво работата на схемите и системите се описва като операции и прехвърляне на данни между регистри.

$$\mathbf{r_{dest} \leftarrow f(\, r_{src1}, r_{src2}, \ldots, r_{srcn}),}$$

където $\mathbf{r_{dest}}$ е регистър, в който се записват данните.

Регистрите $\mathbf{r_{src1}, r_{src2}, \ldots, r_{srcn}}$ съдържат изходните данни.

Примери за операции на регистрово ниво са:

r1 ← 0. В регистър **r1** се записва константа 0.

r1 ← r1. Съдържанието на **r1** се записва обратно в същия регистър.

r2 ← r2 » 3. Съдържанието на **r2** се измества с 3 позиции надясно и се записва отново в r2.

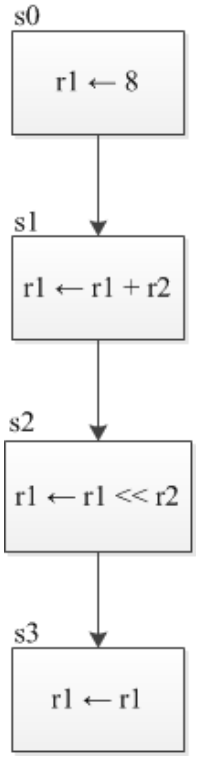
r2 ← r1. Съдържанието на **r1** се прехвърля в r2.

i ← i + 1. Съдържанието на **i** се увеличава с 1-ца и резултата се записва отново в **i**.

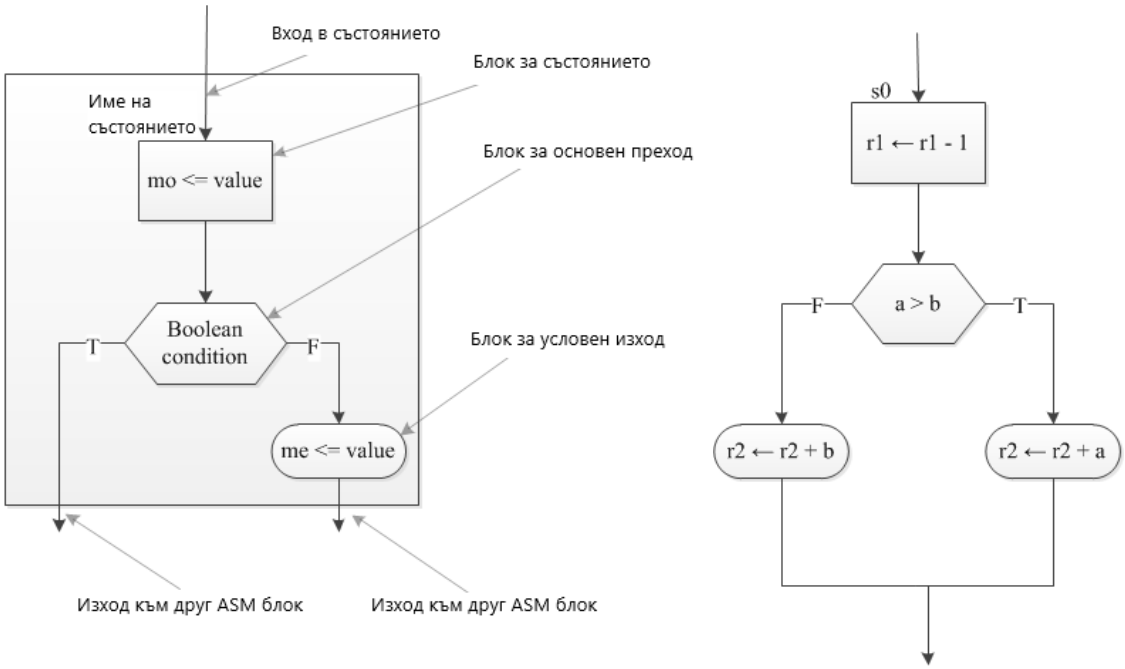
d ← s1 + s2 + s3. Сумата от стойностите в **s1, s2** и **s3** се записва в регистър **d**.

y ← a * a. Стойността в **a** се повдига на квадрат и се записва в **y**.

Последователността от операции и преходите между тях, осъществявани в цифровите схеми се представя с **ASMD (Algorithmic State Machine Diagram)** диаграма (фиг. 4.1). Тази диаграмата е съставена от **ASMD** блокове, които се състоят от: блок за състоянието, указващ текущото състояние на схемата; блокове за условен преход и блокове за условен изход (или условна операция), както е показано на фиг. 4.2.



Фиг. 4.1. ASMD диаграма



Фиг.4.2 . ASMD блок

Регистровите операции се изпълняват синхронно с тактов сигнал. Регистрите, в които се записват резултатите от операциите, се обновяват при напускане на текущия ASMD блок. Например операцията **r ← r – 1** от фиг. 4.2 ще се извърши по следния начин:

r_next <= r_reg – 1

r_reg <= r_next,

по нарастващия фронт на тактовия сигнал, когато се напуска текущия ASMD блок. Наставките **_next** и **_reg** се използват, за да се представи входа и изхода на регистъра.

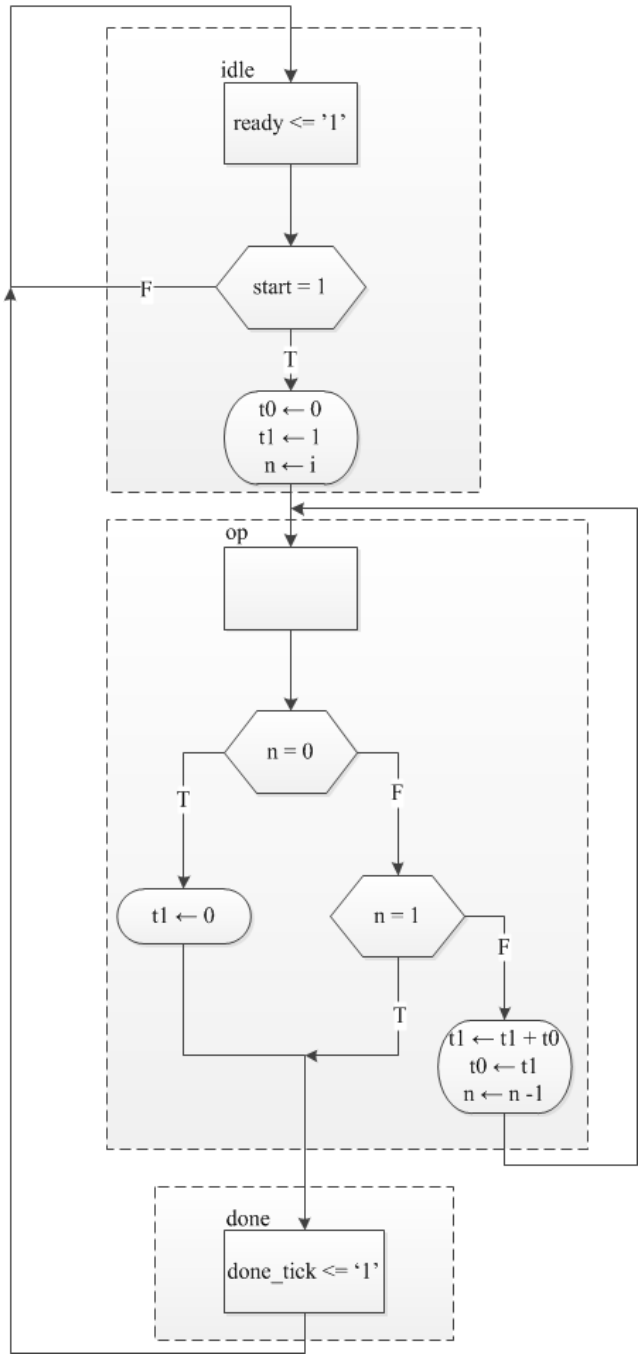
Пример: Моделиране на схема, генерираща числата на Фибоначи.

Числата на Фибоначи се получават по следната формула:

$$fib(i) = \begin{cases} 0 & \text{if } i = 0 \\ 1 & \text{if } i = 1 \\ fib(i-1) + fib(i-2) & \text{if } i > 1 \end{cases}$$

За да се направи модел на регистрово ниво за показаната формула трябва да се използват два регистъра. В тях се съхраняват двете последно намерени числа **fib(i – 1)** и **fib(i – 2)**. Също така е необходим и един индексен регистър. В него се съхранява броят на итерациите.

ASMD диаграмата, описваща работата на схемата за генериране на числата на Фибоначи е представена на фиг. 4.3. Двата регистъра за последно изчислените стойности са означени с **t1** и **t0**, а индексният регистър е означен с **n**. Изходът на схемата, на който последователно се извеждат генерираните числа е означен с **f**. Схемата приема като вход 5-битово число **i**, с което се задава желаният брой итерации, т.е. колко числа на Фибоначи да бъдат генерирани. Командният сигнал **start**, се използва за стартиране работата на схемата. Сигналът за състоянието – **ready**, показва че схемата е в готовност и чака да приеме нови входни данни. Сигналът **done_tick**, се установява в 1-ца за един период на тактовия сигнал, когато моделът приключи работата си. Чрез него установяваме, че моделът е приключил работата си и можем да отчетем резултата.



Фиг.4.3 . ASMD диаграма на генератор на числата на Фибоначи

ASMD диаграмата има три състояния. Състоянието **idle** показва, че схемата е в режим на готовност. Когато се подаде сигнал **start** схемата преминава в състояние **op** и зарежда първоначалните стойности в трите регистъра. Регистрите **t0** и **t1** се зареждат с **0** и **1**, това са числата **fib(0)** и **fib(1).** Желания брой на итерациите **i** се зарежда в регистъра **n**.

Основното изчисление се извършва в състояние **op**, чрез три регистрови операции:

t1 ← t1 + t0

t0 ← t1

n ← n – 1

Чрез първите две операции се изчисляват новите стойности на **t0** и **t1**. Третата операция намалява с 1-ца итерационният индекс. Итерациите спират когато **n** стане 1-ца. Ако първоначалната стойност на **n** е 0, т.е. търси се само **fib(0).**

За разлика от обикновените блок-схеми на алгоритми, операциите в ASMD блоковете се изпълняват паралелно, в рамките на един и същ тактов период. Поставянето и на трите операции в един и същ блок цели намаляване на времето за изчисление.

Новите стойности на **t0** и **t1** се зареждат при напускане на състоянието **op**, т.е. по следващия нарастващ фронт на тактовия сигнал. По този начин в **t0** се съхранява първоначалната стойност на **t1**, а не **t1+ t0**.

VHDL моделът съответстващ на ASMD диаграмата е представен на фиг. 4.4.

Изходът **f** трябва да бъде с достатъчна широчина (бр. битове), за да побере бързо нарастващите числа от генерираната поредица.

II. Практическа част

```
1       library IEEE;
2       use ieee.std_logic_1164.all;
3       use ieee.numeric_std.all;
4
5       entity fib is
6               Port ( clk, reset : in  STD_LOGIC;
7                       start : in  STD_LOGIC;
8                       i : in  STD_LOGIC_VECTOR (4 downto 0);
9                       ready, done_tick : out  STD_LOGIC;
10                      f : out  STD_LOGIC_VECTOR (19 downto 0));
11      end fib;
12
13      architecture arch of fib is
14              type state_type is (idle, op, done);
15              signal state_reg, state_next: state_type;
16              signal t0_reg, t0_next: unsigned(19 downto 0);
17              signal t1_reg, t1_next: unsigned(19 downto 0);
18              signal n_reg, n_next: unsigned(4 downto 0);
```

```
19      -- FSMD state and data registers
20      begin
21              process(clk, reset)
22              begin
23                      if reset = '1' then
24                              state_reg <= idle;
25                              t0_reg <= (others => '0');
26                              t1_reg <= (others => '0');
27                              n_reg <= (others => '0');
28                      elsif (clk'event and clk = '1') then
29                              state_reg <= state_next;
30                              t0_reg <= t0_next;
31                              t1_reg <= t1_next;
32                              n_reg <= n_next;
33                      end if;
34              end process;
35      -- FSMD next – state logic
36              process(state_reg, n_reg, t0_reg, t1_reg, start, i, n_next)
37              begin
38                      ready <= '0';
39                      done_tick <= '0';
40                      state_next <= state_reg;
41                      t0_next <= t0_reg;
42                      t1_next <= t1_reg;
43                      n_next <= n_reg;
44                      case state_reg is
45                              when idle =>
46                                      ready <= '1';
47                                      if start = '1' then
48                                              t0_next <= (others => '0');
49                                              t1_next <= (0 => '1', others => '0');
50                                              n_next <= unsigned(i);
51                                              state_next <= op;
53                                      end if;
54                              when op =>
55                                      if n_reg = 0 then
56                                              t1_next <= (others => '0');
57                                              state_next <= done;
58                                      elsif n_reg = 1 then
59                                              state_next <= done;
60                                      else
61                                              t1_next <= t1_reg + t0_reg;
62                                              t0_next <= t1_reg;
63                                              n_next <= n_reg - 1;
64                                      end if;
65                              when done =>
```

```
66                                  done_tick <= '1';
67                                  state_next <= idle;
68                          end case;
69                  end process;
70      --output
71      f <= std_logic_vector(t1_reg);
72
73      end arch;
```

Фиг. 4.4. VHDL модел на генератора на числата на Фибоначи

III. Задачи за изпълнение

1. Направете съпоставка между VHDL кода и ASMD диаграмата на генератора на числата на Фибоначи.

2. Въведете и симулирайте VHDL модела от фиг. 4.4.

3. Симулирайте работата на създадения VHDL модел. Анализирайте получените резултати.

IV. Въпроси и задачи за самоподготовка

1. Кои са числата на Фибоначи?
2. Кога може да бъде отчетен резултатът при симулация на модела?
3. Какво е регистър?
4. Моделът на фиг. 4.4. е синхронен или асинхронен?
5. Какво се случва когато **n** стане равно на 1 в дадения модел?
6. Какво е краен автомат?